

9. 클로저

- JavaScript의 유효범위 체인을 이용하여 이미 생명 주기가 끝난 외부 함수의 변수를 참조하는 방법
- 외부에서 내부 변수에 접근할 수 있도록 하는 함수입니다
- 클로저를 만들면 클로저 스코프 안에서 클로저를 만든 외부 스코프(Scope)에 항상 접근할 있다

```
{% code lineNumbers="true" %}
```

```
function counter(start) {
  let cnt = start;
  return {
    increment: function() {
      cnt++;
    },
    get: function() {
      return cnt;
    }
  };
}
var foo = new counter(4);
foo.increment();
console.log(foo.get()); // 5
console.log(foo.cnt);    // Uncaught ReferenceError: cnt is not defined
console.log(cnt);        // Uncaught ReferenceError: cnt is not defined
```

```
{% endcode %}
```

- 15 line : foo.cnt는 내부 변수로 참조 할 수 없음 (스코프 범위 밖)
- 16 line : foo 객체의 내부 변수로 참조 할 수 없음 (스코프 범위 밖)

```
foo.hack = function() {
  cnt = 1337;
};
foo.hack();
console.log(foo.get()); // 5
console.log(count);    // 1337
console.log(foo.cnt);   // Uncaught ReferenceError: cnt is not defined
```

- hack 함수는 Counter 함수에 정의 가 되어 있지 않아서 cnt에 설정을 하여도 cnt 값은 변경 되지 않는다. Hack에서 변경 한 cnt 는 Global 변수에 적용 됨

1. 클로저 오남용

1. 함수 내부의 클로저 구현은 함수의 객체가 생성될 때마다 클로저가 생성되는 결과를 가져옵니다. 같은 구동을 하는 클로저가 객체 마다 생성이 된다면 쓸데없이 메모리를 쓸데없이 차지하게 되는데, 이를 클로저의 오남용이라함 -> 성능 문제 (메모리)

```
function MyObject(inputname) {
  this.name = inputname;
  this.getName = function() { // 클로저
    return this.name;
  };
  this.setName = function(rename) { // 클로저
    this.name = rename;
  };
}

var obj = new MyObject("서");
console.log(obj.getName());
```

- **prototype**객체를 이용한 클로저 생성

```
function MyObject(inputname) {
  this.name = inputname;
}

MyObject.prototype.getName = function() {
  return this.name;
};

MyObject.prototype.setName = function(rename) {
  this.name = rename;
};

var obj = new MyObject("서");
console.log(obj.getName());
```

2. Closure를 통해서 외부 함수의 this와 arguments객체를 참조 할 수 없음

```
{% code lineNumbers="true" %}
```

```
function f1(){
  console.log(arguments[0]); // 1
  function f2(){
    console.log("f2 ===== " );
    console.log(arguments[0]); // undefined
  }
  return f2;
}

var exam = f1(1);
exam(); // 1
```

```
{% encode %}
```

- 5 line에서 arguments 객체를 참조 할 수 없음

```
function f1(){
  console.log(arguments[0]); // 1
  var a = arguments[0];
  function f2(){
    console.log(a); // 1
  }
  return f2;
}

var exam = f1(1);
exam();
```

2. Closure를 이용한 캡슐화

캡슐화란 간단하게 말하면 객체의 자료화 행위를 하나로 묶고, 실제 구현 내용을 외부에 감추는 겁니다. 즉, 외부에서 볼 때는 실제 하는 것이 아닌 추상화 되어 보이게 하는 것으로 정보은닉

```
function Gugudan(dan){
  this.maxDan = 3;

  this.calculate = function(){
    for(var i =1; i<=this.maxDan; i++){
      console.log(dan+"*"+i+"="+dan*i);
    }
  }
}

Gugudan.prototype.setMaxDan = function(reset) {
  this.maxDan = reset;
};

var dan5 = new Gugudan(5);
dan5.calculate();
dan5.maxDan=2;
dan5.calculate();
dan5.setMaxDan(4);
dan5.calculate();
```

5*1=5

5*2=10

5*3=15

5*1=5

5*2=10

5*1=5

5*2=10

5*3=15

5*4=20

3. 반복문 에서 Closure 사용 하기

타이머에 설정된 익명 함수는 변수 i에 대한 참조를 들고 있다가 console.log가 호출되는 시점에 i의 값을 사용한다.

console.log가 호출되는 시점에서 for loop는 이미 끝난 상태기 때문에 i 값은 10이 된다

```

for(var i = 0; i < 10; i++) {
  setTimeout(function() {
    console.log(i); // 10 ( 10만 출력됨 )
  }, 100);
}

```

1. 익명 함수로 랩핑

```

for(var i = 0; i < 10; i++) {
  ((e)=> { // function(e) {
    setTimeout(function() {
      console.log(e);
      // 0,1,2,3,4,5,6,8,9
    }, 100);
  })(i);
}

```

2. 랩핑한 익명 함수에서 출력 함수를 반환

```

for(var i = 0; i < 10; i++) {
  setTimeout((function(e) {
    return function() {
      console.log(e);
      // 0,1,2,3,4,5,6,8,9
    }
  })(i), 100);
}

```

3. setTimeout 함수에 세번째 인자를 추가

```

for(var i = 0; i < 10; i++) {
  setTimeout(function(e) {
    console.log(e);
    // 0,1,2,3,4,5,6,8,9
  }, 100, i);
}

```

4. bind를 사용하는 방법

```

for(var i = 0; i < 10; i++) {
  setTimeout(
    console.log.bind(console, i),
    100);
  // 0,1,2,3,4,5,6,8,9
}

```